ARTIFICIAL INTELLIGENCE CSA1701 LAB OBSERVATON
Mohammed Fareeth S A (192511288)

EXPERIMENT 11:

Aim

To implement and solve the classical Map Coloring Problem as a Constraint Satisfaction Problem (CSP) using Backtracking Search in Python to assign valid colors to regions such that no adjacent regions share the same color.

Algorithm

1. Define CSP Components: Set up the variables (regions), domains (available colors), and constraints (adjacency rules).
2. Backtracking Search: Initialize an empty assignment dictionary. Recursively select an unassigned variable and attempt to assign a color from its domain.
3. Consistency Checking: Before making an assignment, check if any neighboring region already possesses the proposed color. If a conflict exists, reject the color.
4. Recursive Exploration & Backtracking: Proceed to the next variable if consistent. If a dead-end is reached (no colors fit), backtrack by removing the last assignment and trying an alternative color.
5. Termination: Return the complete valid assignment dictionary once all variables are successfully colored.

Code:

```python
class MapColoringCSP:
    def __init__(self, variables, domains, constraints):
        self.variables = variables
        self.domains = domains
        self.constraints = constraints

    def backtracking_search(self, assignment=None):
        if assignment is None:
            assignment = {}

        if len(assignment) == len(self.variables):
            return assignment

        unassigned = [v for v in self.variables if v not in assignment]
        var = unassigned[0]

        for color in self.domains[var]:
            if self.is_consistent(var, color, assignment):
                assignment[var] = color

                result = self.backtracking_search(assignment)
                if result is not None:
                    return result

                del assignment[var]

        return None

    def is_consistent(self, var, color, assignment):
        for neighbor in self.constraints[var]:
            if neighbor in assignment and assignment[neighbor] == color:
                return False
        return True

variables = ['WA', 'NT', 'SA', 'Q', 'NSW', 'V', 'T']

domains = {v: ['Red', 'Green', 'Blue'] for v in variables}

constraints = {
    'WA': ['NT', 'SA'],
    'NT': ['WA', 'SA', 'Q'],
    'SA': ['WA', 'NT', 'Q', 'NSW', 'V'],
    'Q': ['NT', 'SA', 'NSW'],
    'NSW': ['Q', 'SA', 'V'],
    'V': ['SA', 'NSW'],
    'T': []
}

csp = MapColoringCSP(variables, domains, constraints)
solution = csp.backtracking_search()

for region, color in solution.items():
```

```
============= RESTART: C:/Users/Admin/OneDrive/Documents/aiexp11.p
WA ---> Red
NT ---> Green
SA ---> Blue
Q ---> Red
NSW ---> Green
V ---> Red
T ---> Red
```

Result

The CSP backtracking algorithm successfully evaluated the constraints across all interconnected regions. Starting from an empty assignment, the program systematically assigned valid colors from the domain while ensuring that no neighboring regions shared the same color.

Conclusion

The program successfully demonstrated the implementation of a Constraint Satisfaction Problem (CSP) using backtracking search. By explicitly managing variables, domains, and constraints, the algorithm efficiently navigated the combinatorial search space to find a conflict-free configuration for the map coloring problem.